

```
import pandas as pd

orders = pd.read_csv("olist_orders_dataset.csv")
reviews = pd.read_csv("olist_order_reviews_dataset.csv")
customers = pd.read_csv("olist_customers_dataset.csv")
products = pd.read_csv("olist_products_dataset.csv")
translation = pd.read_csv("product_category_name_translation.csv")

print(orders.shape)
print(orders.head())
```

[Show hidden output](#)

```
# Join orders with reviews (one review per order)
merged = orders.merge(reviews, on="order_id", how="left")

# Join that with customers (to get the state)
merged = merged.merge(customers, on="customer_id", how="left")

print(merged.shape)
merged.head()
```

[Show hidden output](#)

Next steps: [Generate code with merged](#) [New interactive sheet](#)

```
# Check which order_ids have more than one review
review_counts = reviews["order_id"].value_counts()
duplicated_orders = review_counts[review_counts > 1]
print(duplicated_orders)
```

[Show hidden output](#)

```
# Sort so the newest review comes first, then keep only the first review per order
reviews_clean = reviews.sort_values("review_creation_date", ascending=False)
reviews_clean = reviews_clean.drop_duplicates(subset="order_id", keep="first")

# Redo the merge with the cleaned reviews
merged = orders.merge(reviews_clean, on="order_id", how="left")
merged = merged.merge(customers, on="customer_id", how="left")

print(merged.shape)
```

[Show hidden output](#)

```
# Convert the date columns from text into real dates
merged["order_estimated_delivery_date"] = pd.to_datetime(merged["order_estimated_delivery_date"])
merged["order_delivered_customer_date"] = pd.to_datetime(merged["order_delivered_customer_date"])

# Calculate the difference in days
merged["days_difference"] = (merged["order_estimated_delivery_date"] - merged["order_delivered_customer_date"]).dt.days

print(merged["days_difference"].describe())
```

[Show hidden output](#)

```
# Classify delivery status
def classify_delay(days):
    if pd.isna(days):
        return "Not Delivered"
    elif days >= 0:
        return "On Time"
    elif days >= -5:
        return "Late"
    else:
        return "Super Late"

merged["delivery_status"] = merged["days_difference"].apply(classify_delay)

print(merged["delivery_status"].value_counts())
```

[Show hidden output](#)

```
# Only look at delivered orders (exclude "Not Delivered")
delivered = merged[merged["delivery_status"] != "Not Delivered"]

# For each state, calculate % of orders that are Late or Super Late
state_summary = delivered.groupby("customer_state")["delivery_status"].apply(
    lambda x: (x.isin(["Late", "Super Late"])).mean() * 100
).sort_values(ascending=False)

print(state_summary)
```

```
customer_state
AL    23.929471
MA    19.665272
PI    15.966387
CE    15.324472
SE    15.223881
BA    14.035627
RJ    13.470412
TO    12.773723
PA    12.367865
ES    12.230576
RR    12.195122
MS    11.554922
PB    11.025145
PE    10.797238
RN    10.759494
SC     9.754722
GO     8.175779
RS     7.148204
DF     7.067308
MT     6.772009
SP     5.894555
MG     5.618670
PR     4.996953
AP     4.477612
AM     4.137931
AC     3.750000
RO     2.880658
Name: delivery_status, dtype: float64
```

```
# Average review score by delivery status
review_by_status = merged.groupby("delivery_status")["review_score"].mean()
print(review_by_status)
```

```
delivery_status
Late          3.460482
Not Delivered  1.747098
On Time       4.293916
Super Late    1.785296
Name: review_score, dtype: float64
```

```
import matplotlib.pyplot as plt

# Take the top 10 worst states for a clear chart
top_states = state_summary.head(10)

plt.figure(figsize=(10, 6))
top_states.plot(kind="bar", color="crimson")
plt.title("Top 10 States by Late Delivery Rate")
plt.xlabel("State")
plt.ylabel("% Late or Super Late")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

[Show hidden output](#)

```
order = ["On Time", "Late", "Super Late", "Not Delivered"]
review_by_status_ordered = review_by_status.reindex(order)

plt.figure(figsize=(8, 5))
review_by_status_ordered.plot(kind="bar", color=["green", "orange", "red", "gray"])
plt.title("Average Review Score by Delivery Status")
plt.xlabel("Delivery Status")
plt.ylabel("Average Review Score (1-5)")
plt.xticks(rotation=0)
```

```
plt.tight_layout()
plt.show()
```

[Show hidden output](#)

```
order_items = pd.read_csv("olist_order_items_dataset.csv")

print(order_items.shape)
order_items.head()
```

[Show hidden output](#)

```
# Keep only the first item per order, to avoid duplicating rows again
order_items_clean = order_items.drop_duplicates(subset="order_id", keep="first")

# Link order_id to product_id
merged = merged.merge(order_items_clean[["order_id", "product_id"]], on="order_id", how="left")

# Link product_id to category name
merged = merged.merge(products[["product_id", "product_category_name"]], on="product_id", how="left")

# Translate category name to English
merged = merged.merge(translation, on="product_category_name", how="left")

print(merged.shape)
merged[["product_category_name", "product_category_name_english"]].head()
```

[Show hidden output](#)

```
# Only look at delivered orders (exclude "Not Delivered")
delivered_products = merged[merged["delivery_status"] != "Not Delivered"]

# For each category, calculate % of orders that are Late or Super Late
category_summary = delivered_products.groupby("product_category_name_english")["delivery_status"].apply(
    lambda x: (x.isin(["Late", "Super Late"])).mean() * 100
)

# Only keep categories with a reasonable number of orders, so the % isn't based on just 1-2 orders
category_counts = delivered_products["product_category_name_english"].value_counts()
reliable_categories = category_counts[category_counts >= 100].index

category_summary = category_summary[reliable_categories].sort_values(ascending=False)

print(category_summary.head(10))
```

[Show hidden output](#)

```
plt.figure(figsize=(10, 6))
category_summary.head(10).plot(kind="bar", color="darkorange")
plt.title("Top 10 Product Categories by Late Delivery Rate")
plt.xlabel("Product Category")
plt.ylabel("% Late or Super Late")
plt.xticks(rotation=60, ha="right")
plt.tight_layout()
plt.show()
```

[Show hidden output](#)

```
# Keep only the columns useful for the dashboard, to keep it light
dashboard_data = merged[[
    "order_id",
    "customer_state",
    "order_purchase_timestamp",
    "days_difference",
    "delivery_status",
    "review_score",
    "product_category_name_english"
]]

dashboard_data.to_csv("dashboard_data.csv", index=False)
print("Saved:", dashboard_data.shape)
```

Saved: (99441, 7)

